

Lebanese University
Faculty of Science

FINAL REPORT

Designing Convolutional Networks for Satellite Image Classification

Hand-Design, Transfer Learning and Training-Free Neural Architecture Search on EuroSAT

Prepared by: Emilio Yared

Program: Professional Master's in Artificial Intelligence and Data Engineering (M2)

Course: Scientific English and Communication Skills

Supervisor: Dr. Zeinab Abbas

Date of submission: October 2026

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Zeinab Abbas, for her continuous guidance and constructive feedback on the structure and the language of this report.

I would also like to thank my colleague Joe Missisian, with whom the experimental work of the AIDE500 project “Programming for AI” was carried out, as well as the course instructors for providing the computing resources and laboratory material on which the implementation is based.

Finally, my appreciation goes to the authors of the EuroSAT dataset and of the NAS-Bench-201 benchmark for making their data publicly available, and to the Kaggle platform for providing free access to graphics processing units.

Abstract

Satellite image classification supports land-use monitoring, but the design of an accurate convolutional neural network (CNN) for a new dataset still requires a long process of trial and error. This study compared three approaches for building a classifier for the EuroSAT RGB dataset (27,000 images, ten land-cover classes): the manual design of a CNN trained from scratch, transfer learning from an ImageNet-pretrained ResNet-18, and training-free neural architecture search (NAS) based on the NASWOT score. The NASWOT score was first reproduced on the NAS-Bench-201 benchmark and compared with five search strategies (random search, regularised evolution, local search, REINFORCE and assisted evolution), each executed 200 times. Manual design increased the test accuracy of the baseline CNN from 90.15% to 97.04%, whereas the fully fine-tuned ResNet-18 reached 99.04%, and 95.95% with only 5% of the training data. On NAS-Bench-201, NASWOT matched the published results within 0.1 percentage point in a few seconds, while regularised evolution required 100 trained networks to obtain a further gain of 1.2 points. On EuroSAT, the architecture selected by NASWOT reached 97.43% with twelve times less GPU time than manual design. These findings indicate that training-free search is a practical first step for designing compact models, whereas transfer learning remains the most accurate option when model size is not constrained.

Keywords: convolutional neural networks, satellite imagery, EuroSAT, transfer learning, neural architecture search, NASWOT, zero-cost proxies.

Table of Contents

1 Introduction 1
1.1 Context and Problem Statement 1
1.2 Objectives 1
1.3 Importance of the Project 2
1.4 Structure of the Report 2
2 Background 3
2.1 Convolutional Neural Networks 3
2.2 Optimisation and Regularisation 3
2.3 Transfer Learning 4
2.4 Neural Architecture Search 4
2.5 NAS Benchmarks 5
2.6 Related Work on EuroSAT 5
3 Methodology 6
3.1 Overview 6
3.2 Dataset 6
3.3 Data Preprocessing 7
3.4 Approach 1: Manual Design of a CNN 7
3.5 Approach 2: Transfer Learning 7
3.6 Approach 3: Training-Free Architecture Search 8
3.7 Evaluation Metrics 9
3.8 Hardware and Software Setup 9
4 Results and Analysis 10
4.1 Baseline CNN 10
4.2 Effect of the Architecture 10
4.3 Effect of the Optimiser and the Learning Rate 10
4.4 Effect of Data Augmentation 11
4.5 Transfer Learning 12
4.6 Reproduction of NASWOT on NAS-Bench-201 13
4.7 Comparison of NAS Strategies on NAS-Bench-201 15
4.8 Comparison with Published NAS Methods 16
4.9 Architecture Search on EuroSAT 16
4.10 Final Comparison of All Models 18
4.11 Error Analysis 18
5 Discussion and Recommendations 20
5.1 Interpretation of the Results 20
5.2 Limitations 20
5.3 Recommendations 21
6 Conclusion 22
References 23
Appendices 25
Appendix A: Training Hyperparameters 25
Appendix B: EuroSAT Search Space 25

List of Figures

Figure 1 Example of a NAS-Bench-201 cell 5
Figure 2 Overview of the experimental pipeline 6
Figure 3 Example images of the ten EuroSAT classes 7
Figure 4 Learning curves of the baseline CNN 10
Figure 5 Transfer learning versus amount of training data 12
Figure 6 NASWOT score versus CIFAR-10 accuracy on NAS-Bench-201 14
Figure 7 Accuracy of each search strategy versus search budget 15
Figure 8 Accuracy versus latency of the 50 models 18

List of Tables

Table 1 Summary of the EuroSAT RGB dataset 7
Table 2 Effect of the architecture 10
Table 3 Comparison of optimisers and schedulers 11
Table 4 Effect of data augmentation 12
Table 5 Transfer learning versus amount of data 12
Table 6 Kendall's τ: this study versus the paper 14
Table 7 NASWOT search: this study versus the paper 15
Table 8 Comparison of search strategies on NAS-Bench-201 15
Table 9 Published results of NAS methods on NAS-Bench-201 16
Table 10 Correlation of the zero-cost proxies 16
Table 11 Networks selected on the EuroSAT search space 17
Table 12 Hand-designed CNN versus NASWOT winner 18
Table 13 Final comparison of the representative models 18
Table 14 Main hyperparameters of the two final models 25
Table 15 Factors of the EuroSAT search space 25

List of Abbreviations

AREA	Assisted Regularised Evolution
BN	Batch Normalisation
BOHB	Bayesian Optimisation and HyperBand
CNN	Convolutional Neural Network
DARTS	Differentiable Architecture Search
ENAS	Efficient Neural Architecture Search
F1	Harmonic mean of precision and recall
GPU	Graphics Processing Unit
LS	Local Search
NAS	Neural Architecture Search
NASWOT	Neural Architecture Search Without Training
ReLU	Rectified Linear Unit
REA	Regularised Evolution
RGB	Red, Green and Blue channels
RS	Random Search
RSPS	Random Search with Parameter Sharing
SGD	Stochastic Gradient Descent
SNIP	Single-shot Network Pruning

1 Introduction

1.1 Context and Problem Statement

Earth observation satellites such as Sentinel-2 acquire images of the entire surface of the planet every few days. These images are used to monitor agriculture, urban expansion, deforestation and water resources. Since the volume of data is far too large for manual inspection, the automatic classification of land use and land cover has become an important application of deep learning (Helber et al., 2019).

Convolutional neural networks (CNNs) are currently the standard tool for image classification (Krizhevsky et al., 2012; LeCun et al., 1998). However, the performance of a CNN depends strongly on its architecture, that is, on the number of layers, the size of the kernels, the width of each layer and the connections between them. In practice, this architecture is designed manually: an engineer proposes a network, trains it, observes the results and modifies the design. Each iteration requires a complete training run, which may last from several minutes to several days. Consequently, manual design is slow, expensive and strongly dependent on the experience of the designer.

Two families of methods have been proposed to reduce this effort. The first one, transfer learning, reuses a network that has already been trained on a large dataset such as ImageNet (Deng et al., 2009) and adapts it to the new task (Yosinski et al., 2014). The second one, neural architecture search (NAS), automates the design of the architecture itself (Elsken et al., 2019). Early NAS methods were extremely costly: the original algorithm of Zoph and Le (Zoph & Le, 2017) used 800 GPUs for 28 days. More recently, Mellor et al. (Mellor et al., 2021) proposed NASWOT, a training-free method that scores an architecture at initialisation in less than one second. If such a score is reliable, the cost of architecture design could be reduced from hours to seconds.

1.2 Objectives

The main objective of this project was to compare the three approaches (manual design, transfer learning and training-free NAS) on the same satellite image classification task. More specifically, the objectives were:

1. To build a baseline CNN for the EuroSAT dataset and to improve it systematically by modifying its architecture, its optimiser and its data augmentation strategy.
2. To evaluate transfer learning from an ImageNet-pretrained ResNet-18 with different amounts of training data.

3. To reproduce the NASWOT results of Mellor et al. (Mellor et al., 2021) on the NAS-Bench-201 benchmark (Dong & Yang, 2020), and to compare NASWOT with five other NAS strategies.
4. To apply NASWOT to a search space of CNNs designed for EuroSAT and to compare the resulting network with the manually designed one.

1.3 Importance of the Project

This project is important for two reasons. From a practical point of view, organisations that process satellite images often need models that run on limited hardware, for example on board a satellite or on a field device. Knowing the trade-off between accuracy, model size and design cost helps engineers to select the appropriate approach. From a methodological point of view, NAS papers frequently report results that are difficult to reproduce. An independent reproduction on a public benchmark therefore provides evidence on whether training-free search can be trusted in practice.

1.4 Structure of the Report

The remainder of this report is organised as follows. Chapter 2 presents the necessary background on CNNs, transfer learning and NAS. Chapter 3 describes the dataset, the three approaches and the evaluation protocol. Chapter 4 reports and analyses the results. Chapter 5 discusses their implications and provides recommendations. Finally, Chapter 6 concludes the report. Additional details are provided in the appendices.

2 Background

2.1 Convolutional Neural Networks

A CNN is a neural network that processes images through a sequence of convolutional layers (LeCun et al., 1998). Each convolutional layer applies a set of small filters (kernels), typically of size 3×3 or 5×5, to its input. Each filter detects a local pattern such as an edge, a colour transition or a texture. The output of the convolution is passed through a non-linear activation function, usually the rectified linear unit (ReLU), defined as $\mathrm{ReLU}(x) = \max(0, x)$.

Pooling layers reduce the spatial resolution of the feature maps, so that the deeper layers observe a larger region of the image, and fully connected layers at the end of the network produce a score for each class.

Several components have been introduced to make deep CNNs easier to train. Batch normalisation (BN) normalises the activations of each layer and allows higher learning rates (Ioffe & Szegedy, 2015). Residual (skip) connections, introduced in ResNet (He et al., 2016), add the input of a block to its output; this allows the gradient to flow through very deep networks. Finally, the use of several small kernels instead of a single large kernel was popularised by the VGG networks (Simonyan & Zisserman, 2015), since two 3×3 convolutions cover the same receptive field as one 5×5 convolution with fewer parameters.

2.2 Optimisation and Regularisation

The weights of a CNN are learnt by minimising a loss function, generally the cross-entropy, with a gradient-based optimiser. Stochastic gradient descent (SGD) with momentum updates the weights in the direction opposite to the gradient. Adam (Kingma & Ba, 2015) adapts the learning rate of each parameter individually, and AdamW (Loshchilov & Hutter, 2019) decouples the weight decay from this adaptation. The learning rate itself can be decreased during training by a scheduler, for example a step decay, a cosine annealing (Loshchilov & Hutter, 2017) or a one-cycle policy (Smith & Topin, 2019).

When the training set is small compared to the capacity of the network, the model may memorise the training images instead of learning general features; this phenomenon is called overfitting. Data augmentation reduces overfitting by applying random transformations (flips, crops, rotations or colour changes) to the training images. More advanced methods such as mixup (Zhang et al., 2018) and CutMix (Yun et al., 2019) combine two images and their labels into a single training example.

2.3 Transfer Learning

Transfer learning consists in reusing a network trained on a large source dataset for a different target task. Yosinski et al. (Yosinski et al., 2014) demonstrated that the first layers of a CNN learn generic features (edges, textures), whereas the last layers learn features that are specific to the source task. Two strategies are commonly used:

- **Feature extraction:** the pretrained layers are frozen and only a new classification layer is trained.
- **Fine-tuning:** some or all of the pretrained layers are trained further on the target data, usually with a smaller learning rate.

This is particularly effective when the target dataset is small.

2.4 Neural Architecture Search

NAS automates the design of neural network architectures. According to the survey of Elsken et al. (Elsken et al., 2019), every NAS method can be described by three components:

- **Search space:** the set of architectures that can be represented, for example all combinations of a fixed number of layers and operations.
- **Search strategy:** the algorithm that explores the search space, such as random sampling, evolution or reinforcement learning.
- **Performance estimation strategy:** the method used to estimate the quality of a candidate architecture. The most reliable but most expensive strategy is to train the candidate completely.

2.4.1 Training-Based Search Strategies

The first NAS algorithms trained every candidate architecture. Zoph and Le (Zoph & Le, 2017) used a recurrent controller network trained with the REINFORCE algorithm (Williams, 1992): the controller proposes an architecture, the architecture is trained, and its accuracy is used as a reward to update the controller. Real et al. (Real et al., 2019) proposed regularised evolution (REA), in which a population of architectures is improved by mutation, and the oldest individuals are removed from the population. Bayesian optimisation methods such as BOHB (Falkner et al., 2018) build a model of the accuracy as a function of the architecture. White et al. (White et al., 2021) showed that a simple local search, which repeatedly moves to the best neighbouring architecture, is highly competitive on NAS benchmarks.

2.4.2 Weight-Sharing Methods

To reduce the cost of training each candidate, weight-sharing methods train a single "super-network" that contains all the candidate architectures. ENAS (Pham et al., 2018), DARTS (Liu

et al., 2019), GDAS (Dong & Yang, 2019a), SETN (Dong & Yang, 2019b) and random search with parameter sharing (RSPS) (Li & Talwalkar, 2019) belong to this family. These methods are faster but unstable (Dong & Yang, 2020).

2.4.3 Training-Free Search and Zero-Cost Proxies

Training-free methods estimate the quality of an architecture without any training, by computing a score at initialisation. Abdelfattah et al. (Abdelfattah et al., 2021) evaluated several of these "zero-cost proxies", including SNIP (Lee et al., 2019), SynFlow (Tanaka et al., 2020) and the gradient norm. Mellor et al. (Mellor et al., 2021) proposed the NASWOT score, which is based on the activation patterns of the ReLU units and is described in detail in Section 3.5.

2.5 NAS Benchmarks

Comparing NAS methods is difficult, because each method is usually evaluated on a different search space with a different training procedure. NAS-Bench-201 (Dong & Yang, 2020) addresses this problem. It defines a search space of 15,625 cell-based architectures and provides the accuracy of every architecture after a complete training on CIFAR-10, CIFAR-100 and ImageNet16-120. As a result, a search strategy can be evaluated by simply looking up the accuracy of the architectures that it selects, without training any network. Each cell is a small directed graph with four nodes and six edges, and each edge applies one of five operations, as illustrated in Figure 1.

2.6 Related Work on EuroSAT

Helber et al. (Helber et al., 2019) introduced the EuroSAT dataset and reported an accuracy of 98.57% on the RGB version with a fine-tuned ResNet-50. To the best of our knowledge, training-free architecture search has not been evaluated on EuroSAT in previous work, which motivates the third part of this project.

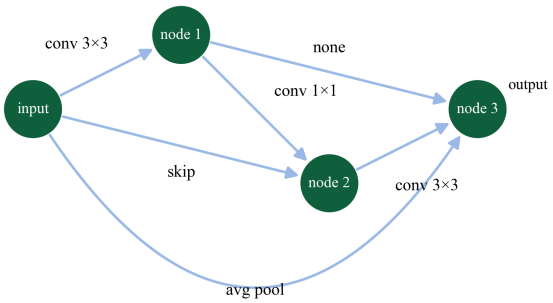


Figure 1. Example of a NAS-Bench-201 cell. Each of the six edges applies one of five operations (none, skip connection, 1×1 convolution, 3×3 convolution or 3×3 average pooling), which gives $5^6 = 15{,}625$ architectures.

3 Methodology

3.1 Overview

The project was organised as a pipeline of experiments, as illustrated in Figure 2. All the experiments used the same dataset split and the same test set, so that the three approaches could be compared fairly.



Figure 2. Overview of the experimental pipeline. The three approaches share the same data split and are evaluated on the same 4,050 test images.

3.2 Dataset

The EuroSAT RGB dataset (Helber et al., 2019) was obtained from Kaggle. It contains 27,000 images of 64×64 pixels extracted from Sentinel-2 satellite data, labelled into ten land-use and land-cover classes. Table 1 summarises its main characteristics, and Figure 3 shows one example per class.

Table 1. Summary of the EuroSAT RGB dataset.

Characteristic	Value
Number of images	27,000
Image size	64 × 64 pixels, 3 channels (RGB)
Number of classes	10
Images per class	2,000 to 3,000
Training / validation / test split	18,900 / 4,050 / 4,050 (70% / 15% / 15%)
Split strategy	Stratified by class, fixed random seed (42)
Source	Sentinel-2 satellite, 13 European countries



Figure 3. One example image from each of the ten EuroSAT classes.

The dataset is approximately balanced (2,000 to 3,000 images per class).

3.3 Data Preprocessing

The split into training, validation and test sets was computed once in the first notebook (NB0) and saved in a file that was reused by all the other notebooks. The mean and standard deviation of each colour channel were computed on the training set only and were used to normalise all images. All the design decisions were based on the validation set, and the test set was used only once per model.

3.4 Approach 1: Manual Design of a CNN

3.4.1 Baseline Network

The baseline network consisted of three convolutional blocks followed by two fully connected layers. Each block contained a 3×3 convolution with padding, a ReLU activation and a 2×2 max-pooling layer. The number of channels was doubled at each block (32, 64 and 128), and the classifier contained a hidden layer of 256 units. The network had 2.19 million parameters. It was trained for 30 epochs with SGD (learning rate 0.01, momentum 0.9) and a batch size of 64.

3.4.2 Systematic Study

Starting from the baseline, one factor was modified at a time, while all the others were kept constant. Three groups of experiments were conducted:

- **Architecture:** number of convolutional blocks (2 to 5), kernel size (3×3, 5×5, 7×7), padding, down-sampling by pooling or by strided convolution, number of fully connected layers, and batch normalisation.
- **Optimisation:** three optimisers (SGD, Adam, AdamW), three learning rates (10^{-2}, 10^{-3}, 10^{-4}) and three learning-rate schedulers (step decay, cosine annealing and one-cycle).
- **Data augmentation:** horizontal and vertical flips, random crops, affine transformations, colour jitter, mixup and CutMix.

The best option of each group was retained for the following group, and the final model combined the best architecture, the best optimiser and the best augmentation strategy. This final model is referred to as the “hand-designed CNN” in the rest of the report.

3.5 Approach 2: Transfer Learning

A ResNet-18 (He et al., 2016) pretrained on ImageNet was used as the source network. The images were resized to 128×128 pixels and normalised with the ImageNet statistics. The final layer was replaced by a dropout layer followed by a linear layer with ten outputs. Three strategies were compared with a CNN trained from scratch:

- **Frozen:** all pretrained layers were frozen and only the new classifier was trained.
- **Partial fine-tuning:** the last residual stage (layer4) and the classifier were trained.
- **Full fine-tuning:** all layers were trained, with a learning rate of 10^{-4} for the pretrained layers and 10^{-3} for the classifier.

In order to study the effect of the amount of data, each strategy was trained with 5%, 10%, 25% and 100% of the training set.

3.6 Approach 3: Training-Free Architecture Search

3.6.1 The NASWOT Score

The NASWOT score (Mellor et al., 2021) is computed on an untrained network and a single mini-batch of B images. For each image i, the state of every ReLU unit (active or inactive) is recorded, which gives a binary code c_i. Two images with the same code fall into the same linear region of the network, which means that the network cannot easily separate them. The kernel matrix K_H is defined as:

$$K_H[i, j] = N_A - d_H(c_i, c_j)$$

where N_A is the number of ReLU units and d_H is the Hamming distance between the two codes. The score of the network is the logarithm of the determinant of this matrix:

$$s = \log|K_H|$$

A high score indicates that the images of the mini-batch are mapped to different linear regions; according to Mellor et al., such networks tend to reach a higher accuracy after training. The score was implemented with forward hooks on every ReLU layer.

3.6.2 Reproduction on NAS-Bench-201

Before using the score on EuroSAT, the implementation was validated on NAS-Bench-201, whose 15,625 architectures were rebuilt in PyTorch. Each architecture was then scored with a mini-batch of 128 CIFAR-10 images. The following quantities were compared with the paper: the Kendall rank correlation between the score and the final accuracy, and the accuracy of the network selected by NASWOT when N = 10, 100 or 1,000 architectures are sampled.

3.6.3 Comparison with Other Search Strategies

Five other search strategies were executed on NAS-Bench-201, using the pre-computed accuracies as the result of the training:

- **Random Search (RS):** architectures are sampled uniformly and the best one is kept.

- **Regularised Evolution (REA):** a population of 20 architectures evolves by tournament selection (sample size 5) and single-edge mutation; the oldest individual is removed at each step (Real et al., 2019).
- **Local Search (LS):** all neighbours of the current architecture (24 architectures that differ by one edge) are evaluated, and the search moves to the best one; a random restart is performed at a local optimum (White et al., 2021).
- **REINFORCE:** one categorical distribution per edge is learnt by policy gradient, with a moving-average baseline (Williams, 1992).
- **Assisted REA (AREA):** NASWOT scores 1,000 random architectures and the 20 best ones form the initial population of REA (Mellor et al., 2021).

The cost of each strategy was measured by the number of architectures that had to be trained (the search budget), from 10 to 300. Each strategy was executed 200 times with different random seeds. The search used the CIFAR-10 validation accuracy (Dong & Yang, 2020).

3.6.4 Search on EuroSAT

A search space adapted to EuroSAT was defined using the same factors as in the manual study: 2 to 5 convolutional blocks, a kernel size of 3, 5 or 7 chosen independently for each block, a width of 16, 32 or 64 channels, batch normalisation or not, down-sampling by pooling or by stride, and residual connections or not. This space contains 8,640 different CNNs. First, the reliability of the proxies was measured on 51 random networks trained for five epochs. Then, 1,000 random networks were scored with NASWOT, and the three best ones were trained for five epochs. Finally, the winner was trained with the same recipe as the hand-designed CNN. Three networks selected by SynFlow and three random networks were trained for comparison.

3.7 Evaluation Metrics

The models were evaluated by their test accuracy, their macro F1-score (the average over the ten classes of the harmonic mean of precision and recall) and their confusion matrix. Their cost was measured by the number of parameters and the inference latency in milliseconds per image. The reliability of a proxy was measured by Kendall's τ and Spearman's ρ, two rank correlations between the proxy score and the trained accuracy, and the cost of a search was measured by the GPU time or the number of architectures trained.

3.8 Hardware and Software Setup

All the training experiments were implemented in Python 3.11 with PyTorch and torchvision. They were executed on Kaggle with a single NVIDIA T4 GPU (16 GB). The search-strategy comparison on NAS-Bench-201 required only table look-ups and was executed on a laptop CPU in 11 seconds. The main hyperparameters are listed in Appendix A.

4 Results and Analysis

4.1 Baseline CNN

Figure 4 presents the learning curves of the baseline network. The training loss decreased continuously, whereas the validation loss reached its minimum after approximately ten epochs and then increased. This behaviour indicates that the network started to overfit the training data. The baseline reached a validation accuracy of 91.11% and a test accuracy of 90.15%, with a generalisation gap of 8.9 percentage points between the training and validation accuracies.

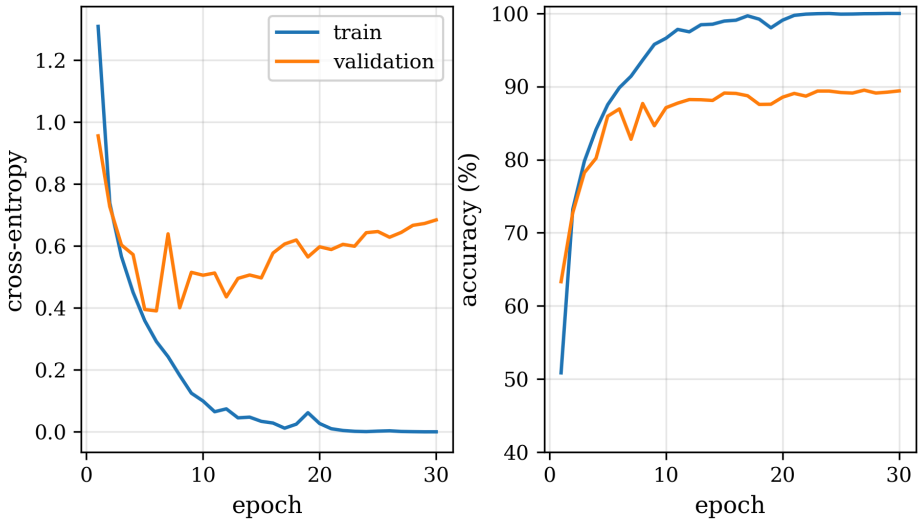


Figure 4. Learning curves of the baseline CNN (loss and accuracy on the training and validation sets).

4.2 Effect of the Architecture

Table 2 summarises the architecture study. Increasing the depth from three to five blocks improved the validation accuracy from 91.11% to 93.04%, while reducing the number of parameters from 2.19 to 1.24 million, since the deeper network has smaller feature maps before the classifier. In contrast, larger kernels degraded the performance: the 7×7 network reached only 79.95%. Replacing pooling by strided convolutions also reduced the accuracy (84.37%). Batch normalisation provided a moderate improvement (+0.79 point). When combined, five blocks of 3×3 convolutions with batch normalisation achieved 94.52%, which is 3.4 points above the baseline.

4.3 Effect of the Optimiser and the Learning Rate

Table 3 presents the results of the optimisation study, which was performed on the combined architecture. With a constant learning rate, Adam with a learning rate of 10^{-3} obtained the

Table 2. Effect of the architecture on the validation accuracy (SGD, 30 epochs, no augmentation).

Variant	Parameters	Val. accuracy (%)	Gap (pt)
Baseline (3 blocks, 3×3)	2.19 M	91.11	8.87
2 blocks	4.22 M	86.74	13.21
4 blocks	1.44 M	92.37	7.32
5 blocks	1.24 M	93.04	6.17
Kernel 5×5	2.36 M	89.16	10.81
Kernel 7×7	2.61 M	79.95	15.87
Valid padding	1.28 M	89.21	10.52
Strided convolution	2.19 M	84.37	15.43
One fully connected layer	0.18 M	90.74	9.21
Batch normalisation	2.19 M	91.90	7.68
Combined (5 blocks + BN)	**1.24 M**	**94.52**	**4.89**

highest validation accuracy (94.15%) and reached 90% after only six epochs. However, its final accuracy (92.22%) was lower than its best accuracy, which indicates that the training oscillated at the end. Adding a learning-rate scheduler solved this problem: both the step decay and the cosine annealing reached 95.85%. The cosine scheduler was retained, because its final accuracy was equal to its best accuracy.

4.4 Effect of Data Augmentation

Table 4 shows that geometric augmentations were beneficial. Flips alone increased the validation accuracy from 95.85% to 96.30% and reduced the generalisation gap from 3.9 to 1.9 points.

Table 3. Comparison of optimisers, learning rates and schedulers (combined architecture).

Configuration	Best val. (%)	Final val. (%)	Epochs to 90%
SGD, lr = 10^{-2}	93.28	93.16	10
SGD, lr = 10^{-3}	93.31	93.21	9
SGD, lr = 10^{-4}	90.52	90.52	23
Adam, lr = 10^{-2}	93.04	91.61	12
Adam, lr = 10^{-3}	94.15	92.22	6
AdamW, lr = 10^{-3}	94.00	94.00	11
Adam + step decay	95.85	95.61	9
Adam + cosine annealing	**95.85**	**95.85**	**6**
Adam + one-cycle	95.53	95.53	8

Table 4. Effect of data augmentation (combined architecture, Adam with cosine annealing).

Augmentation	Best val. (%)	Gap (pt)
None	95.85	3.90
Flips	96.30	1.85
Flips + crop	96.44	1.43
Flips + crop + affine	96.10	0.96
Flips + crop + affine + colour jitter	95.65	0.99
Flips + crop + mixup	**96.52**	**1.21**
Flips + crop + CutMix	95.19	0.91

This result can be explained by the nature of satellite images: unlike photographs, they have no natural "up" direction, so a flipped image is still a realistic example. Random crops and mixup provided small additional gains, and the best combination (flips, crops and mixup) reached 96.52%. In contrast, colour jitter and CutMix decreased the accuracy, probably because colour is an important cue for distinguishing classes such as Forest, SeaLake and AnnualCrop.

On the test set, the final hand-designed CNN obtained an accuracy of 97.04%, which represents an improvement of 6.9 percentage points over the baseline, with 43% fewer parameters.

4.5 Transfer Learning

Table 5 and Figure 5 present the test accuracy of the transfer learning strategies for different amounts of training data. Full fine-tuning achieved the highest accuracy for every data fraction, with a maximum of 99.04% on the complete training set. Moreover, with only 5% of the data (947 images), the fine-tuned network reached 95.95%, whereas the CNN trained from scratch reached only 87.85%. The frozen network saturated at approximately 90%, which confirms that the ImageNet features of the last layers are not optimal for satellite images. Unfreezing only the last stage (layer4) recovered most of the gain.

The fine-tuned network also converged much faster: it reached 90% validation accuracy after a single epoch, whereas the scratch network required six epochs. However, this performance

Table 5. Test accuracy of transfer learning as a function of the fraction of training data.

Method (test accuracy, %)	5%	10%	25%	100%
CNN from scratch	87.85	91.88	94.91	96.64
ResNet-18, frozen	87.51	88.59	89.80	90.44
ResNet-18, layer4 fine-tuned	95.11	95.73	96.89	97.90
ResNet-18, fully fine-tuned	**95.95**	**96.86**	**97.80**	**99.04**

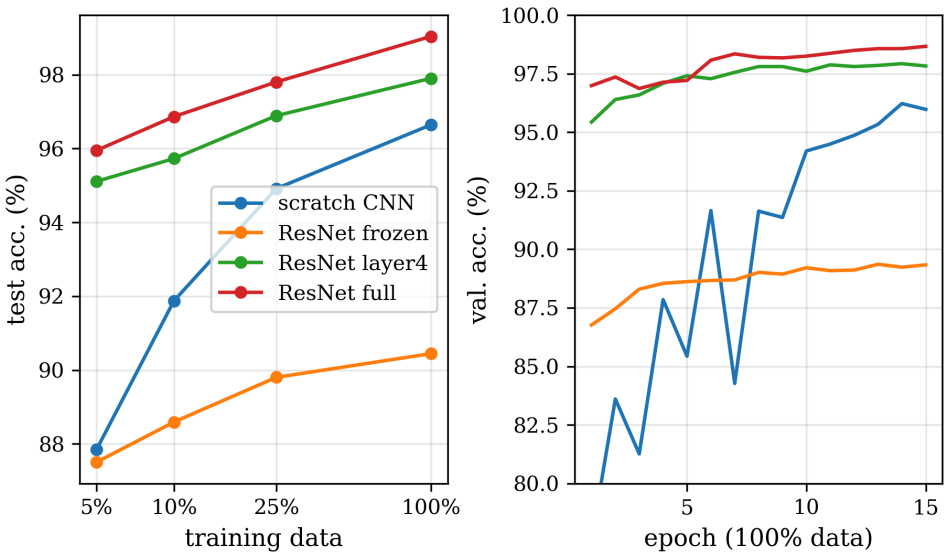


Figure 5. Test accuracy of the four strategies as a function of the amount of training data.

has a cost, since ResNet-18 contains 11.2 million parameters, which is nine times more than the hand-designed CNN.

4.6 Reproduction of NASWOT on NAS-Bench-201

4.6.1 Correctness of the Implementation

The re-implemented networks had exactly the official number of parameters for all 300 architectures that were checked, and the best architecture of the benchmark had a CIFAR-10 test accuracy of 94.37%, which matches the official value. This confirms that the search space was reproduced correctly.

4.6.2 Correlation Between Score and Accuracy

Table 6 compares the Kendall correlation obtained in this study with the values reported by Mellor et al. The correlations were slightly higher than the published values, by 0.03 to 0.05. This small difference can be attributed to the random choice of the mini-batch and of the sampled architectures. Figure 6 shows the relation between the score and the accuracy for all 15,625 architectures.

Table 6. Kendall's τ between the NASWOT score and the final validation accuracy (1,000 architectures). *Approximated with CIFAR-100 images at 16×16 pixels.

Dataset	This study	Mellor et al. (2021)	Difference
CIFAR-10	0.602	0.574	+0.028
CIFAR-100	0.640	0.611	+0.029
ImageNet16-120*	0.600	0.549	+0.051

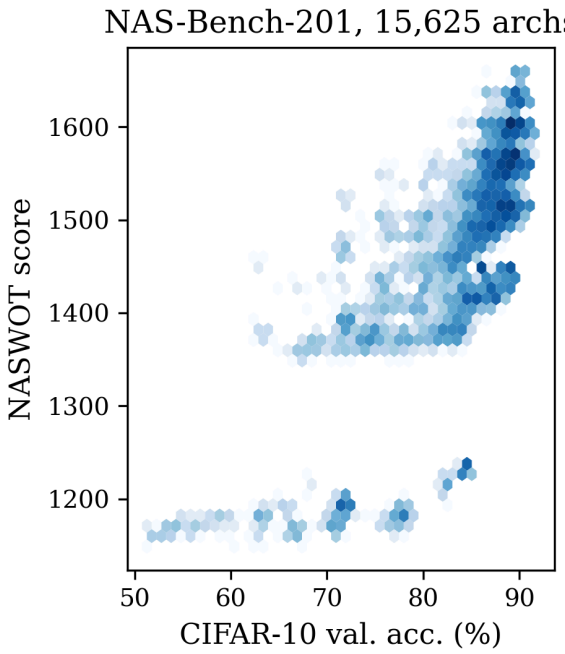


Figure 6. NASWOT score at initialisation versus CIFAR-10 validation accuracy after training, for the 15,625 architectures of NAS-Bench-201.

The scatter plot shows a clear positive relation, but also a large dispersion among the best architectures. Consequently, NASWOT is able to discard poor architectures, but it cannot reliably distinguish the very best architecture from other good ones. The score was also stable with respect to the mini-batch: the Kendall correlation between the rankings obtained with two different mini-batches was 0.99. Scoring one architecture required 160 milliseconds.

4.6.3 Search Results

Table 7 compares the accuracy of the architecture selected by NASWOT with the values reported in the paper. For $N = 10$ and $N = 100$, the differences on CIFAR-10 and CIFAR-100 were below 0.1 percentage point. Therefore, the NASWOT search was considered successfully reproduced.

Table 7. Test accuracy (%) of the selected architecture, mean ± standard deviation over 500 runs. *Approximated with CIFAR-100 images.

Method	CIFAR-10 test	CIFAR-100 test	ImageNet16 test*
NASWOT (N=10), this study	92.39 ± 1.2	68.63 ± 2.31	41.29 ± 3.94
NASWOT (N=10), paper	92.44 ± 1.13	68.62 ± 2.04	41.31 ± 4.11
NASWOT (N=100), this study	92.83 ± 0.94	69.52 ± 1.67	43.18 ± 2.95
NASWOT (N=100), paper	92.81 ± 0.99	69.48 ± 1.7	43.1 ± 3.16
Random, this study	87.04 ± 12.94	61.36 ± 12.17	33.54 ± 9.52
Random, paper	86.61 ± 13.46	60.83 ± 12.58	33.13 ± 9.66

Table 8. Test accuracy (%) of the architecture selected by each strategy on NAS-Bench-201 (mean ± standard deviation over 200 runs; search on the CIFAR-10 validation set).

Strategy	Trained nets	CIFAR-10	CIFAR-100	ImageNet16
Random Search (RS)	100	93.76 ± 0.32	71.12 ± 0.99	44.71 ± 1.03
Regularised Evolution (REA)	100	94.12 ± 0.27	72.36 ± 1.1	45.83 ± 0.89
Local Search (LS)	100	93.98 ± 0.35	71.73 ± 1.27	45.34 ± 1.15
REINFORCE	100	93.86 ± 0.26	71.48 ± 1.03	44.99 ± 1.04
Assisted REA (AREA)	100	94.11 ± 0.29	72.44 ± 1.07	45.87 ± 0.86
NASWOT (N = 10)	0	92.36 ± 1.27	68.52 ± 2.43	41.26 ± 4.1
NASWOT (N = 100)	0	92.85 ± 0.88	69.56 ± 1.6	43.21 ± 2.76
NASWOT (N = 1000)	0	92.93 ± 0.96	69.81 ± 1.33	44.1 ± 2.52
Best architecture of the space	–	94.37	73.22	46.71

4.7 Comparison of NAS Strategies on NAS-Bench-201

Table 8 presents the accuracy obtained by the six search strategies after 100 trained architectures, together with the training-free NASWOT. Figure 7 shows how the accuracy of each strategy evolves with the search budget.

Several observations can be made. First, all training-based strategies outperformed NASWOT when they were allowed to train 100 architectures: regularised evolution and assisted evolution reached 94.1% on CIFAR-10, which is only 0.25 point below the best architecture of the space. Second, NASWOT without any training (92.9%) was equivalent to a random search

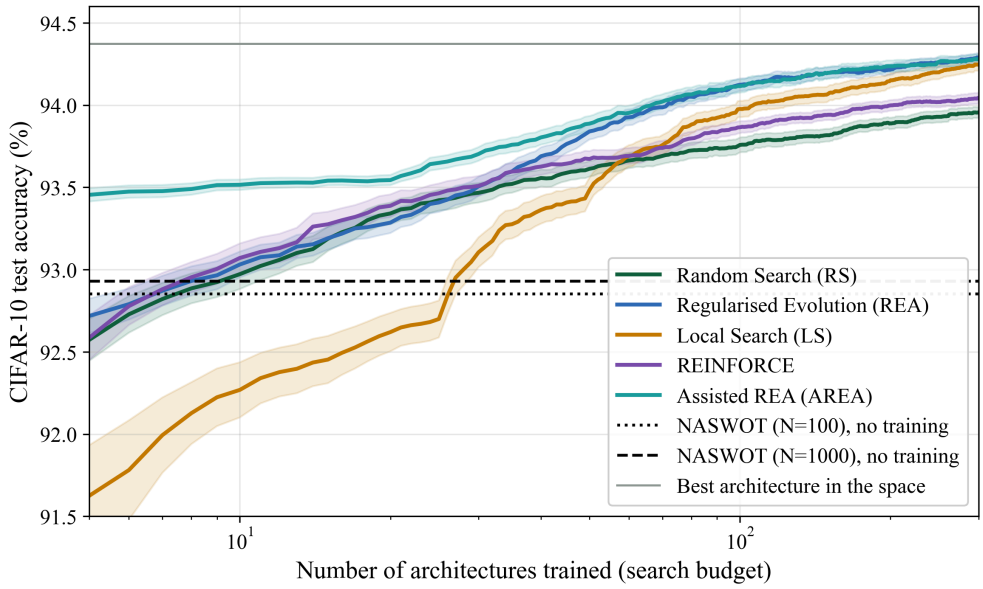


Figure 7. CIFAR-10 test accuracy of the best architecture found as a function of the number of architectures trained (mean of 200 runs; shaded area: 95% confidence interval).

with approximately ten trained architectures. Third, local search was the weakest strategy with a small budget, because it spends its first evaluations on the neighbours of a single random architecture; however, it caught up with evolution after approximately 150 evaluations, which is consistent with White et al. (White et al., 2021).

The most interesting result concerns the combination of both approaches. As shown in Figure 7, assisted REA, which uses NASWOT to select its initial population, was clearly superior with small budgets: with only ten trained architectures, it reached 93.5% on CIFAR-10, against 93.0% for regular evolution and random search. As the budget increased, the advantage disappeared, since the evolution itself found good architectures.

In the context of the whole search space, the distribution of accuracies is highly skewed: most architectures reach between 85% and 94%, but a non-negligible number of architectures fail completely. A random choice (86.6% on average) frequently selects such a poor architecture, whereas NASWOT avoids them almost systematically.

4.8 Comparison with Published NAS Methods

Table 9 compares NASWOT with the results published for ten other NAS methods on the same benchmark (Dong & Yang, 2020; Mellor et al., 2021). Weight-sharing methods showed a highly variable performance. In particular, DARTS and ENAS converged to a degenerate architecture with only 54.3% accuracy on CIFAR-10. In contrast, NASWOT reached 92.4% to 93.0% in 3 to 306 seconds, that is, 40 to 4,000 times faster than the training-based methods, which require 12,000 seconds.

4.9 Architecture Search on EuroSAT

4.9.1 Reliability of the Proxies

Table 10 presents the Spearman correlation between each proxy and the accuracy of 51 EuroSAT networks trained for five epochs. The correlations were weaker than on NAS-Bench-201. NASWOT obtained a moderate correlation (0.43), and the gradient norm and SNIP obtained slightly higher values. In contrast, SynFlow and the number of parameters were not informative on this search space, which shows that larger networks are not necessarily better on EuroSAT.

4.9.2 Search Results

Scoring 1,000 random networks with NASWOT required 153 seconds. Table 11 compares the three networks selected by NASWOT with those selected by SynFlow and with three random networks, after five epochs of training. The NASWOT selection obtained the highest mean accuracy (90.8%), against 88.0% for SynFlow and 83.3% for the random networks.

Table 9. Published results of NAS methods on NAS-Bench-201 (Mellor et al., 2021, Table 2; search times on a single GTX 1080 Ti GPU).

Method	Family	Search time (s)	CIFAR-10 test (%)
REA	Training-based	12,000	93.92 ± 0.30
Random search	Training-based	12,000	93.70 ± 0.36
REINFORCE	Training-based	12,000	93.85 ± 0.37
BOHB	Training-based	12,000	93.61 ± 0.52
RSPS	Weight sharing	7,587	87.66 ± 1.69
DARTS-V1	Weight sharing	10,890	54.30 ± 0.00
DARTS-V2	Weight sharing	29,902	54.30 ± 0.00
GDAS	Weight sharing	28,926	93.51 ± 0.13
SETN	Weight sharing	31,010	86.19 ± 4.63
ENAS	Weight sharing	13,315	54.30 ± 0.00
NASWOT (N = 10)	Training-free	3	92.44 ± 1.13
NASWOT (N = 100)	Training-free	30	92.81 ± 0.99
NASWOT (N = 1000)	Training-free	306	92.96 ± 0.81

The best network selected by NASWOT contained five blocks with kernel sizes 5, 3, 3, 3 and 3, a width of 64 channels, batch normalisation, max-pooling and residual connections. After a

Table 10. Rank correlation between each zero-cost proxy and the trained accuracy on the two search spaces.

Proxy	Spearman ρ (EuroSAT)	Spearman ρ (NAS-Bench-201)
NASWOT	0.43	0.81
SynFlow	0.09	0.80
SNIP	0.51	0.65
Gradient norm	0.56	0.65
Number of parameters	−0.13	0.76

Table 11. Five-epoch validation accuracy of the networks selected on the EuroSAT search space.

Selection method (top-3)	Mean val. accuracy, 5 epochs (%)	Mean parameters
NASWOT	**90.76**	**3.30 M**
SynFlow	87.96	5.23 M
Random	83.32	6.89 M
Hand-designed CNN (reference)	87.7	1.24 M

Table 12. Comparison between the hand-designed CNN and the network found by NASWOT (same training recipe, retrained in NB6.2).

Criterion	Hand-designed CNN	NASWOT winner
Test accuracy (%)	96.72	**97.43**
Parameters	**1.24 M**	1.99 M
Inference latency (ms)	**0.76**	1.31
GPU time spent on the design	203 min	**17 min**

complete training with the same recipe as the hand-designed CNN, it reached a test accuracy of 97.43%. Table 12 compares it with the hand-designed network.

The searched network was 0.7 point more accurate than the hand-designed one, and the design required twelve times less GPU time. Interestingly, the search rediscovered the main conclusions of the manual study (deep networks, small kernels and batch normalisation) and additionally selected residual connections, which had not been tested manually.

4.10 Final Comparison of All Models

All 50 checkpoints were reloaded and evaluated on the same 4,050 test images. Table 13 presents the four representative models, and Figure 8 shows the trade-off between accuracy and latency.

No single model was the best on every criterion. The fine-tuned ResNet-18 was the most accurate model, whereas the scratch CNNs were the fastest and the smallest. The network found by NASWOT occupied an intermediate position.

4.11 Error Analysis

The confusion matrices of the hand-designed CNN and of the fine-tuned ResNet-18 were also analysed. The hand-designed CNN misclassified 120 of the 4,050 test images, and the ResNet-18 misclassified 39. In both cases, most errors occurred between the agricultural classes (PermanentCrop, HerbaceousVegetation, AnnualCrop and Pasture), whose images are similar green textures at a resolution of 64×64 pixels. Furthermore, the two models made different

Table 13. Final comparison of the representative models on the common test set (NB7).

Model	Accuracy (%)	Macro F1 (%)	Parameters	Latency (ms)
Baseline CNN	90.15	89.76	2.19 M	**0.34**
Hand-designed CNN	97.04	96.97	**1.24 M**	0.79
NASWOT winner	97.43	97.36	1.99 M	1.27
ResNet-18, fully fine-tuned	**99.04**	**98.99**	11.18 M	2.23

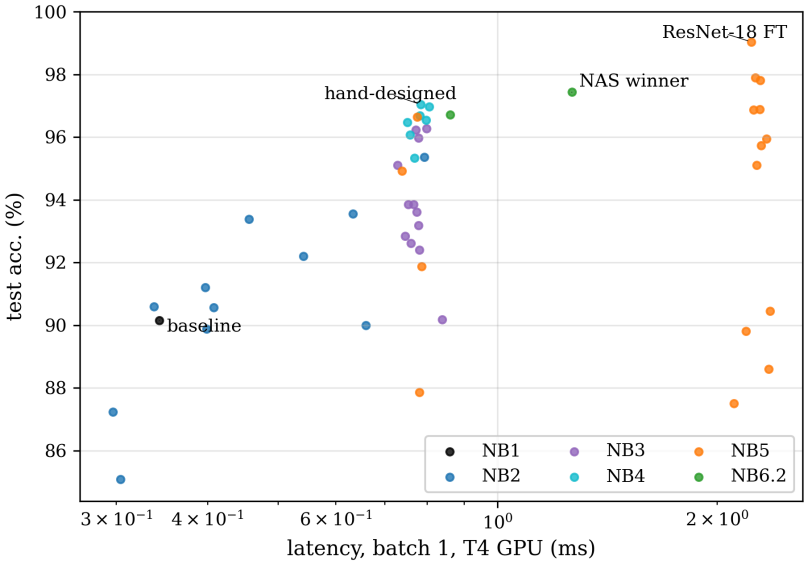


Figure 8. Test accuracy versus inference latency for the 50 trained models.

errors, and only 14 images were misclassified by all three main models. A visual inspection of these images revealed that they mostly contain mixed land uses, for example a field crossed by a road, for which the label itself is ambiguous.

5 Discussion and Recommendations

5.1 Interpretation of the Results

The results demonstrate that each of the three approaches has a distinct domain of application. Manual design produced a compact and fast network (1.24 million parameters, 0.79 ms per image), but it required more than three hours of GPU time and a large number of decisions. Transfer learning produced the most accurate model and was by far the best option with limited data; however, its size and latency make it less suitable for embedded deployment. Training-free search produced, in a few minutes, a network that was more accurate than the manual design, which indicates that the score captures useful information about the trainability of an architecture.

The comparison on NAS-Bench-201 clarifies the role of NASWOT. It is not a replacement for training-based search, since evolution with 100 trained networks consistently found better architectures. Nevertheless, NASWOT is an excellent filter: it avoids the poor regions of the search space at almost no cost, and it is most useful when combined with a training-based method, as shown by assisted evolution with small budgets.

In the real-world context of satellite monitoring, these findings mean that the choice of approach should depend on the deployment constraints. When the model is executed in a data centre, a fine-tuned pretrained network is the most appropriate choice. When the model must run on limited hardware, a training-free search followed by a short training-based refinement offers a good compromise between accuracy and design cost.

5.2 Limitations

The following limitations should be considered when interpreting the results:

- **Single seed:** each configuration of the manual study was trained only once. Repeated runs vary by approximately 0.3 to 1 point, so only large effects can be considered conclusive.
- **Short training for the EuroSAT proxies:** the correlation study on EuroSAT used a five-epoch training as ground truth, which is a noisy estimate of the final accuracy.
- **ImageNet16-120 approximation:** this dataset was not available on Kaggle, and the scores were computed with resized CIFAR-100 images.
- **Simulated search cost:** on NAS-Bench-201, the training of an architecture was replaced by a table look-up, and the cost was measured by the number of architectures trained rather than by actual GPU time.

- **Single dataset:** the conclusions on EuroSAT have not been verified on other remote sensing datasets or on multispectral images.

5.3 Recommendations

Based on these results, the following recommendations are proposed:

1. For satellite image classification without strict size constraints, fine-tune a pretrained network rather than designing a CNN from scratch, especially when fewer than a few thousand labelled images are available.
2. Use orientation-invariant augmentations (flips and rotations) for overhead imagery, and avoid strong colour transformations.
3. Use a training-free score such as NASWOT to filter the search space before any training, and then train only the best candidates or use them to initialise an evolutionary search.
4. Validate a zero-cost proxy on a small sample of trained networks before relying on it in a new search space, since its correlation may be weaker than on public benchmarks.
5. In future work, repeat the experiments with several random seeds, extend the study to the 13 spectral bands of EuroSAT, and evaluate hardware-aware search with a latency constraint.

6 Conclusion

This project aimed to compare three approaches for building a convolutional network that classifies satellite images: manual design, transfer learning and training-free neural architecture search. A baseline CNN was improved step by step through its architecture, its optimiser and its data augmentation, an ImageNet-pretrained ResNet-18 was fine-tuned with different amounts of data, and the NASWOT score was reproduced on NAS-Bench-201, compared with five search strategies and finally applied to EuroSAT.

The manual study increased the test accuracy from 90.15% to 97.04%, and the fine-tuned ResNet-18 obtained the highest accuracy of 99.04%. The reproduction confirmed the published NASWOT results within 0.1 percentage point, and the comparison of search strategies showed that NASWOT is most valuable as a fast filter or as an initialisation for evolutionary search. On EuroSAT, the network found by NASWOT reached 97.43% with twelve times less design time than the manual approach.

In summary, training-free architecture search constitutes a practical and inexpensive first step for the design of compact models, while transfer learning remains the most accurate solution when the size of the model is not constrained.

References

Abdelfattah, M. S., Mehrotra, A., Dudziak, Ł., & Lane, N. D. (2021). Zero-Cost Proxies for Lightweight NAS. *International Conference on Learning Representations (ICLR)*.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 248–255.

Dong, X., & Yang, Y. (2019b). One-Shot Neural Architecture Search via Self-Evaluated Template Network. *IEEE/CVF International Conference on Computer Vision (ICCV)*, 3681–3690.

Dong, X., & Yang, Y. (2019a). Searching for a Robust Neural Architecture in Four GPU Hours. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 1761–1770.

Dong, X., & Yang, Y. (2020). NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search. *International Conference on Learning Representations (ICLR)*.

Elsken, T., Metzen, J. H., & Hutter, F. (2019). Neural Architecture Search: A Survey. *Journal of Machine Learning Research*, *20*(55), 1–21.

Falkner, S., Klein, A., & Hutter, F. (2018). BOHB: Robust and Efficient Hyperparameter Optimization at Scale. *International Conference on Machine Learning (ICML)*, 1437–1446.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

Helber, P., Bischke, B., Dengel, A., & Borth, D. (2019). EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, *12*(7), 2217–2226.

Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *International Conference on Machine Learning (ICML)*, 448–456.

Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 1097–1105.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, *86*(11), 2278–2324.

Lee, N., Ajanthan, T., & Torr, P. H. S. (2019). SNIP: Single-shot Network Pruning based on Connection Sensitivity. *International Conference on Learning Representations (ICLR)*.

Li, L., & Talwalkar, A. (2019). Random Search and Reproducibility for Neural Architecture Search. *Conference on Uncertainty in Artificial Intelligence (UAI)*, 367–377.

Liu, H., Simonyan, K., & Yang, Y. (2019). DARTS: Differentiable Architecture Search. *International Conference on Learning Representations (ICLR)*.

Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic Gradient Descent with Warm Restarts. *International Conference on Learning Representations (ICLR)*.

Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. *International Conference on Learning Representations (ICLR)*.

Mellor, J., Turner, J., Storkey, A., & Crowley, E. J. (2021). Neural Architecture Search without Training. *International Conference on Machine Learning (ICML)*, 7588–7598.

Pham, H., Guan, M., Zoph, B., Le, Q., & Dean, J. (2018). Efficient Neural Architecture Search via Parameter Sharing. *International Conference on Machine Learning (ICML)*, 4095–4104.

Real, E., Aggarwal, A., Huang, Y., & Le, Q. V. (2019). Regularized Evolution for Image Classifier Architecture Search. *Proceedings of the AAAI Conference on Artificial Intelligence*, *33*, 4780–4789.

Simonyan, K., & Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. *International Conference on Learning Representations (ICLR)*.

Smith, L. N., & Topin, N. (2019). Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, *11006*.

Tanaka, H., Kunin, D., Yamins, D. L. K., & Ganguli, S. (2020). Pruning Neural Networks without any Data by Iteratively Conserving Synaptic Flow. *Advances in Neural Information Processing Systems (NeurIPS)*.

White, C., Nolen, S., & Savani, Y. (2021). Local Search is State of the Art for Neural Architecture Search Benchmarks. *Conference on Uncertainty in Artificial Intelligence (UAI)*, 1052–1062.

Williams, R. J. (1992). Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, *8*(3), 229–256.

Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How Transferable Are Features in Deep Neural Networks?. *Advances in Neural Information Processing Systems (NeurIPS)*, 3320–3328.

Yun, S., Han, D., Oh, S. J., Chun, S., Choe, J., & Yoo, Y. (2019). CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features. *IEEE/CVF International Conference on Computer Vision (ICCV)*, 6023–6032.

Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). mixup: Beyond Empirical Risk Minimization. *International Conference on Learning Representations (ICLR)*.

Zoph, B., & Le, Q. V. (2017). Neural Architecture Search with Reinforcement Learning. *International Conference on Learning Representations (ICLR)*.

Appendices

Appendix A: Training Hyperparameters

Table 14. Main hyperparameters of the two final models.

Hyperparameter	Hand-designed CNN	ResNet-18 (fine-tuned)
Input size	64 × 64	128 × 128
Optimiser	Adam	Adam
Learning rate	10^{-3}	10^{-4} (backbone), 10^{-3} (head)
Scheduler	Cosine annealing	Cosine annealing
Batch size	64	64
Epochs	30	15 (100% of data) to 60 (5%)
Augmentation	Flips, crop, mixup	Flips, crop
Loss function	Cross-entropy	Cross-entropy
Model selection	Best validation accuracy	Best validation accuracy

Appendix B: EuroSAT Search Space

Table 15. Factors of the EuroSAT search space used for the architecture search.

Factor	Possible values
Number of convolutional blocks	2, 3, 4 or 5
Kernel size (chosen per block)	3 × 3, 5 × 5 or 7 × 7
Width (channels of the first block)	16, 32 or 64
Batch normalisation	Yes or no
Down-sampling	Max-pooling or strided convolution
Residual connections	Yes or no
Total number of architectures	8,640